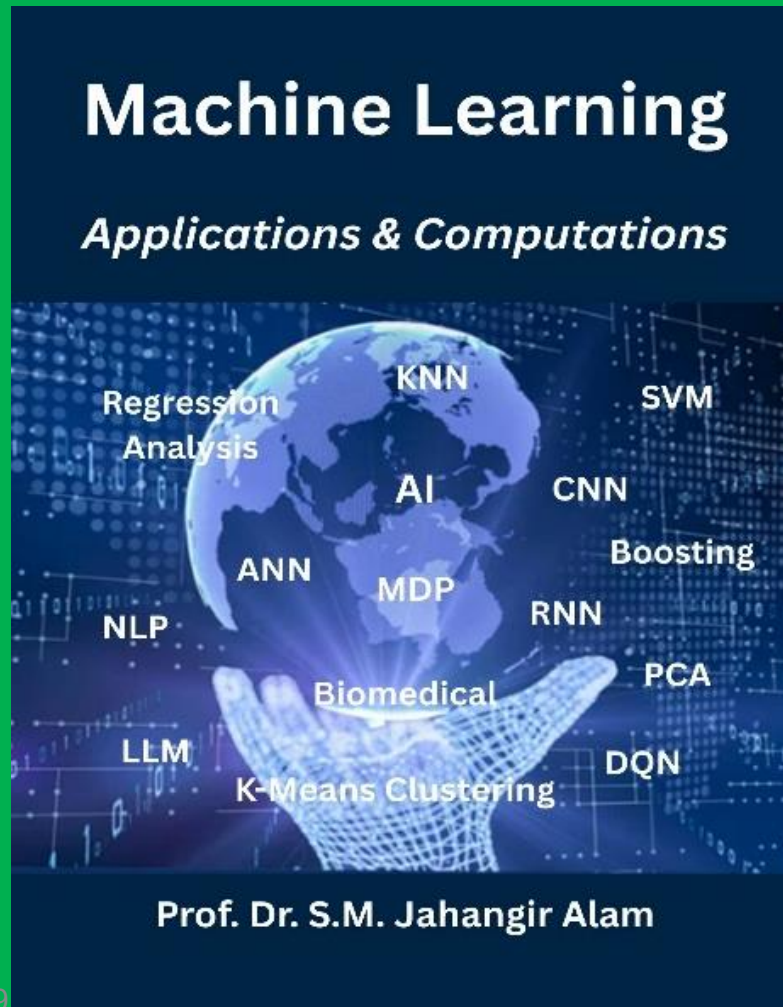


Lecture 3.1

Neural Networks (NN)



Instructor Name:
Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST



Neural Networks (NN)

- ANN is a **computational model inspired by human brain**.
- Consists of **layers of nodes (neurons)** that learn patterns from data.
- Basic neural network with input, hidden, and output layers.
- Each neuron connects to all neurons in next layer (fully connected).
- Works well for tabular data, simple classification, and regression.
- Input layer → receives features
- Hidden layers → transform data through weighted connections and activation functions
- Output layer → produces final predictions



Neural Networks (NN)

NN

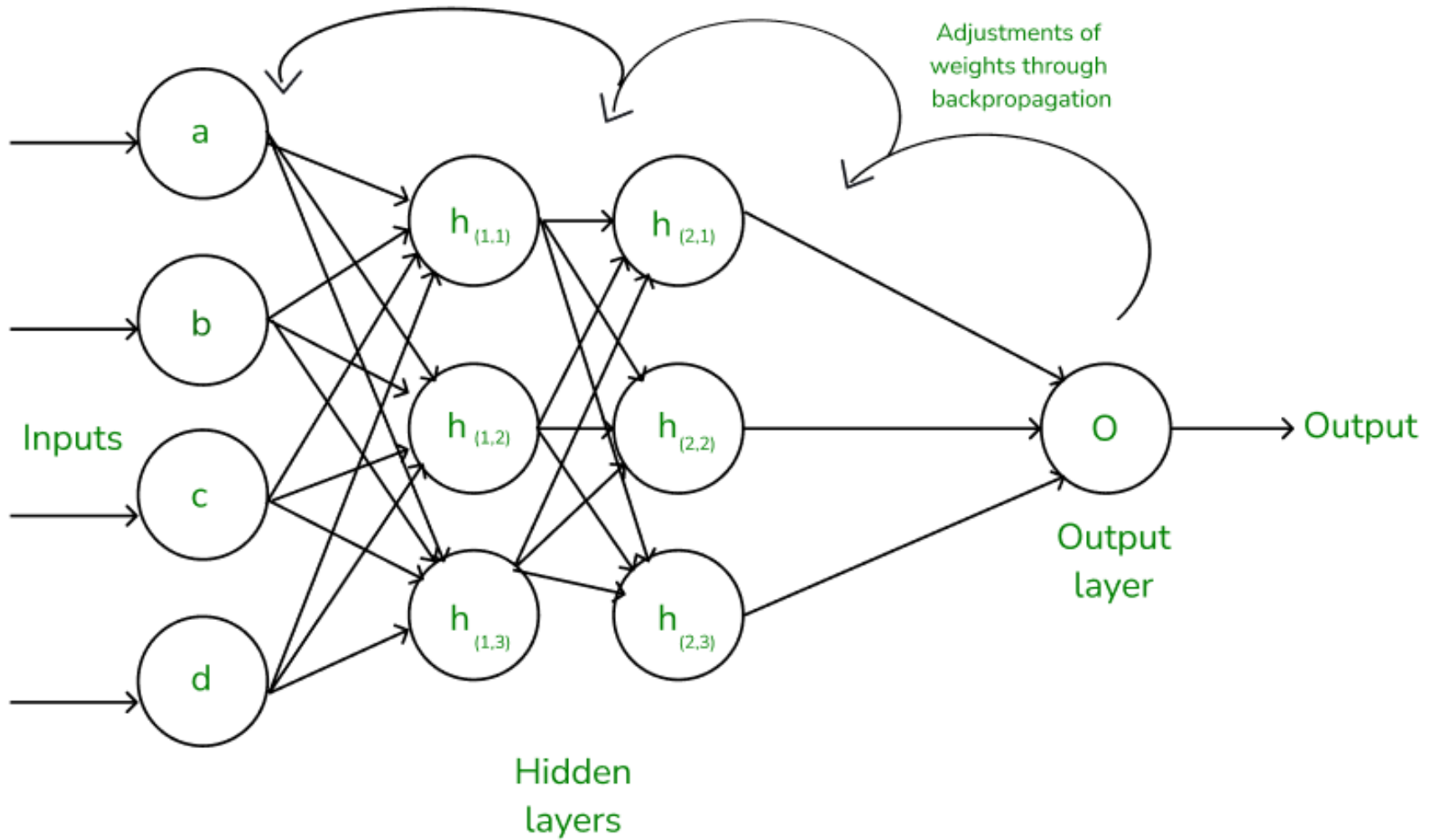
*= input + weighted connections + activation
+ training via backpropagation
→ output prediction.*

Applications of NN

- Image recognition (CNN)
- Speech recognition (RNN, LSTM)
- Natural Language Processing (Transformers, LLM)
- Medical diagnosis
- Finance (stock prediction, fraud detection)



Structure of NN





Structure of NN

Structure of ANN

1. **Input Layer** — receives the features
2. **Hidden Layers** — extract features and patterns
3. **Output Layer** — produces final result (classification, regression, etc.)

Learning in ANN

- Adjust weights and biases to minimize error using algorithms like **gradient descent**.
- Backpropagation updates weights based on the **error signal**.



Structure of NN

1. Neuron (Node)

- Each neuron computes:

$$z = \sum (w_i \cdot x_i) + b$$
$$y = f(z)$$

Where, x_i : input features, w_i : weights (learned), b : bias term;
 $f(z)$: activation function (e.g., sigmoid, ReLU, tanh).

2. Layers

- Input Layer $\rightarrow$ raw features (e.g., pixels of an image, medical data values).
- Hidden Layer(s) $\rightarrow$ multiple neurons that learn feature interactions.
- Output Layer $\rightarrow$ final decision (e.g., classification probability, regression value).



Training an NN

1. Forward Propagation

- Pass inputs $\rightarrow$ hidden $\rightarrow$ output
- Compute prediction

2. Loss Function

- Measure error (e.g., MSE for regression, cross-entropy for classification)

3. Backpropagation

- Compute gradient of loss wrt weights using chain rule
- Adjust weights with Gradient Descent

$$w := w - \eta \cdot \frac{\partial L}{\partial w}$$

- Where, η = learning rate

4. Repeat until convergence



Comparison Table:

Model	Input Type	Key Feature	Best For
NN	Tabular / general	Fully connected layers	Basic ML tasks
CNN	Grid data (images)	Convolutions + pooling	Image, video, vision
RNN	Sequential data	Recurrence (memory of past)	Text, speech, time-series
Deep Learning	Any (complex data)	Multiple hidden layers	Advanced AI (NLP, vision, RL)



Activation Functions

- Activation functions determine the output of a neural network node (neuron) given an input or set of inputs.
- They introduce non-linearity into the network, enabling it to learn complex patterns.

Choosing Activation Functions:

- Hidden layers: ReLU (default), Leaky ReLU, ELU

Output layer:

- Binary classification → Sigmoid
- Multi-class classification → Softmax
- Regression → Linear (no activation)



Activation Functions

Visual Intuition

- **Sigmoid:** S-shaped, squashes values to $(0,1)$
- **Tanh:** S-shaped, squashes values to $(-1,1)$
- **ReLU:** Linear for positive, zero for negative
- **Leaky ReLU:** Linear for positive, small slope for negative

Why Activation Functions Are Important

- Without non-linearity, a neural network with multiple layers would behave like a single-layer linear model.
- Activation functions allow the network to:
 - i. Learn complex relationships
 - ii. Solve classification and regression problems
 - iii. Control the output range

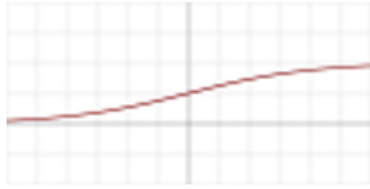
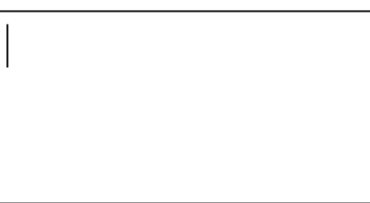


Common Activation Functions:

Function	Formula	Range	Use Case	Notes
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	(0,1)	Binary classification	Saturates for large
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1,1)	Hidden layers	Zero-centered → better than sigmoid
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Hidden layers in deep nets	Simple, fast; can die (0 gradient)
Leaky ReLU	$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{if } x \leq 0 \end{cases}$	$(-\infty, \infty)$	Hidden layers	Avoids dying ReLU problem
Softmax	$\sigma(x) = \frac{e^x}{\sum_j e^{x_j}}$	(0,1), $\sum = 1$	Multi-class classification	Outputs probability distribution
ELU	$\sigma(x) = \begin{cases} x & x > 0 \\ a(e^x - 1) & x \leq 0 \end{cases}$	$(-\alpha, \infty)$	Hidden layers	Smooth gradient, faster learning



Common Activation Functions:

Name	Plot	Function	Range; Use
Identity		$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{if } x \leq 0 \end{cases}$	
Binary step		$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	
Logistic, sigmoid, or soft step		$\sigma(x) = \frac{1}{1 + e^{-x}}$	(0,1); Binary classification
Hard Sigmoid		$f(x) = \begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x > 3 \\ \frac{x}{6} + \frac{1}{2} & \text{otherwise} \end{cases}$	
<u>Softmax</u>		$\sigma(x) = \frac{e^x}{\sum_j e^{x_j}}$	(0,1), sum = 1; Multi-class classification



Common Activation Functions:

Hyperbolic tangent (tanh)		$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$; Hidden layers
Hard tanh		$\tanh(x) = \begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	
Tanh Shrink		$f(x) = x - \tanh(x)$	
Soft Shrink		$f(x) = \begin{cases} x - 1 & \text{if } x > 1 \\ x + 1 & \text{if } x < -1 \\ 0 & \text{otherwise} \end{cases}$	
Hard Shrink		$f(x) = \begin{cases} x & \text{if } x > 1 \\ x & \text{if } x < -1 \\ 0 & \text{otherwise} \end{cases}$	

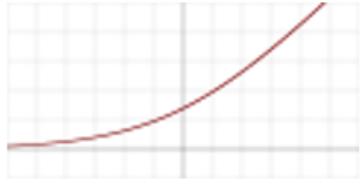
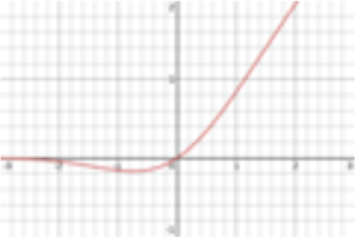


Common Activation Functions:

HardSwish		$f(x) = \begin{cases} 0 & \text{if } x < -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	
Soboleva modified hyperbolic tangent (smht)		$\text{smht}(x) = \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$	
Softsign		$f(x) = \frac{x}{1+ x }$	
Rectified linear unit (ReLU)		$f(x)^+ = \max(0, x)$ $= \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$	$[0, \infty);$ Hidden layers in deep nets
Gaussian Error Linear Unit (GELU)		$\frac{1}{2}x \left(\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \right) = x\phi(x)$ Or, $\frac{1}{2}x \left(1 + \tanh\left(\frac{2}{\sqrt{\pi}}\right)(x + ax^3) \right)$	

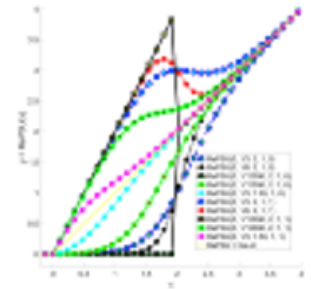
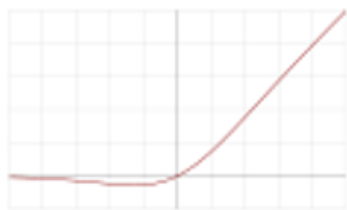
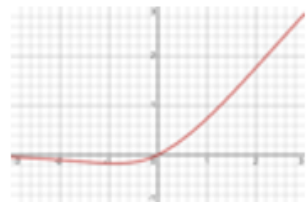
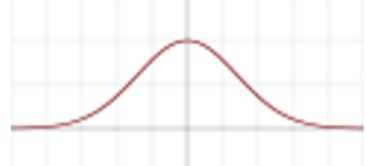


Common Activation Functions:

Softplus		$\frac{1}{\beta} \ln(1 + e^{\beta x})$	
Exponential linear unit (ELU)		$\sigma(x) = \begin{cases} x & x > 0 \\ a(e^x - 1) & x \leq 0 \end{cases}$	$(-\alpha, \infty);$ Hidden layers
<u>Softsign</u>		$f(x) = \frac{x}{1+ x }$	
Rectified linear unit (ReLU)		$f(x)^+ = \max(0, x)$ $= \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$	$[0, \infty);$ Hidden layers in deep nets
Gaussian Error Linear Unit (GELU)		$\frac{1}{2} x \left(\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \right) = x \Phi(x)$ Or, $\frac{1}{2} x \left(1 + \tanh \left(\frac{2}{\sqrt{\pi}} \right) (x + ax^3) \right)$	



Common Activation Functions:

Rectified Parametric Sigmoid Units (flexible, 5 parameters)		$a(2x1_{\{x \geq \Lambda\}} - g_{\Lambda, \sigma, \mu, \beta}(x)) + (1 - \alpha)g_{\Lambda, \sigma, \mu, \beta}(x)$ Where, $g_{\Lambda, \sigma, \mu, \beta}(x) = \frac{(x - \Lambda)1_{\{x \geq \Lambda\}}}{1 + e^{-\text{sgn}(x - \mu)\left(\frac{ x - \mu }{\sigma}\right)^\beta}}$	
Sigmoid linear unit (<u>SiLU</u>, Sigmoid shrinkage, <u>SiL</u>, or Swish-1		$\sigma(x) = \frac{x}{1 + e^{-x}}$	
Exponential Linear Sigmoid <u>SquasHing</u> (<u>ELiSH</u>)		$f(x) = \begin{cases} \frac{e^x - 1}{1 + e^{-x}} & \text{if } x < 0 \\ \frac{x}{1 + e^{-x}} & \text{if } x \geq 0 \end{cases}$	
Gaussian		$f(x) = e^{-x^2}$	



Common Activation Functions

3.1##Activation Functions in neural network. A clear and practical overview of the main types of activation functions used in neural networks, with explanations, equations, and when to use each.

1. Linear Activation Equation: $f(x) = x$

Use: Regression tasks (last layer only)

Issue: No non-linearity $\rightarrow$ entire network acts like a **linear model** (no matter how many layers)

2. Sigmoid (Logistic) Function Equation: $\sigma(x) = \frac{1}{1+e^{-x}}$

Output: (0, 1)

Use: Binary classification (in the **output layer**)

Issues:

- Vanishing gradients for large/small x
- Not zero-centered



Common Activation Functions:

3. Tanh (Hyperbolic Tangent) Equation:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Output: $(-1, 1)$

Use:

- Hidden layers (instead of sigmoid)
- Centered at 0

Issue: Still suffers from vanishing gradient



Common Activation Functions

4. ReLU (Rectified Linear Unit) Equation:

$$f(x) = \max(0, x)$$

Output: $[0, \infty)$

Use: Default for hidden layers in deep neural networks

Issue: Dying ReLU: Neurons can get stuck at 0

5. Leaky ReLU Equation:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

with $\alpha = 0.01$

Use: Fixes dying ReLU problem (allows small gradient when $x < 0$)



Common Activation Functions

6. Parametric ReLU (PReLU)

Like Leaky ReLU, but α is learned during training.

7. ELU (Exponential Linear Unit) Equation:

$$\sigma(x) = \begin{cases} x & x > 0 \\ a(e^x - 1) & x \leq 0 \end{cases}$$

Use:

- Performs better than ReLU on some tasks
- More complex, slightly slower to compute



Common Activation Functions:

8. Softmax (Multiclass Classification Output) Equation:

$$\sigma(x) = \frac{e^x}{\sum_j e^{x_j}}$$

Use:

- Final layer of multiclass classifiers
- Converts logits to probabilities that sum to 1



Common Activation Functions

3.2##Artificial Neural Networks (ANN): Basic concepts, perceptron example.
Considering the basic concepts of Artificial Neural Networks (ANN) and then do a Perceptron example step-by-step using simple binary inputs.

Neuron (Perceptron)

- The **basic unit** of a neural network.
- Computes:

$$y = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$

where:

- ✓ x_i = input
- ✓ w_i = weight
- ✓ b = bias
- ✓ f = activation function (e.g., step, sigmoid, ReLU)



Common Activation Functions

Perceptron Example (Binary Classifier)

Goal: Learn AND logic gate

Initial Setup

- Activation function: **Step function**

$$f(z) = \begin{cases} 1 & z = \omega \cdot x + b \geq 0 \\ 0 & z < 0 \end{cases}$$

- Initial weights: $w_1 = 0, w_2 = 0$
- bias $b = 0$
- Learning rate: $\eta = 1$

x_1	x_2	Output y (Target)
0	0	0
0	1	0
1	0	0
1	1	1

Training dataset (we'll iterate in this order):

(x_1, x_2)	Target y (Target)
(0, 0)	0
(0, 1)	0
(1, 0)	0
(1, 1)	1



Common Activation Functions

$$z = \omega_1 x_1 + \omega_2 x_2 + b$$

$$\therefore y_{pred} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$Error = y - \hat{y}$$

Training (Epoch 1)

We use the perceptron update rule:

$$w_i := w_i + \eta(y - \hat{y})x_i, b := b + \eta(y - \hat{y})$$



Common Activation Functions

Epoch 1:

- Initial weights: $w_1 = 0, w_2 = 0$
- bias $b = 0$
- Learning rate: $\eta = 1$

(x_1, x_2)	Target y (Target)
$(0, 0)$	0
$(0, 1)$	0
$(1, 0)$	0
$(1, 1)$	1

(x_1, x_2)	y	$z = \omega_1 x_1 + \omega_2 x_2 + b$	$\hat{y}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$	$Error = y - \hat{y}$
$(0, 0)$	0	$0 \cdot 0 + 0 \cdot 0 + 0 = 0$	1 (<i>wrong</i>)	$0 - 1 = -1$
Update: $w_1 := 0 + 1 \cdot (-1) \cdot 0 = 0$			$w_2:$ $= 0 + 1 \cdot (-1) \cdot 0 = 0$	$b := 0 + 1 \cdot (-1) = -1$



Common Activation Functions:

Test Final Model:

Input	$z = x_1 + x_2 - 1$	Output
(0, 0)	-1	0
(0, 1)	0	1 (No)
(1, 0)	0	1 (No)
(1, 1)	1	1 (Yes)

→ Still not perfect;
more epochs will fix it.



Common Activation Functions

(x_1, x_2)	y	$z = \omega_1 x_1 + \omega_2 x_2 + b$	$\hat{y}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$	$Error = y - \hat{y}$
$(0, 1)$	0	$0 \cdot 0 + 0 \cdot 1 + (-1) = -1$	$\hat{y} = 0$ (Correct) (target = 0) $\rightarrow$ ok $\rightarrow$ No update	$0 - 0 = 0$
$(1, 0)$	0	$0 \cdot 1 + 0 \cdot 0 + (-1) = -1$	$\hat{y} = 0$ (Correct) (target = 0) $\rightarrow$ ok $\rightarrow$ No update	$0 - 0 = 0$
$(1, 1)$	1	$0 \cdot 1 + 0 \cdot 1 + (-1) = -1$	0 (Wrong)	$1 - 0 = 1$
Update: $w_1 := 0 + 1 \cdot (1) \cdot 1 = 1$			$w_2 := 0 + 1 \cdot (1) \cdot 1 = 1$	$b := -1 + 1 \cdot (1) = 0$

End Epoch 1: $w_1 = 1, w_2 = 1, b = 0$.



Common Activation Functions

Epoch 2:

End Epoch 1: $w_1 = 1, w_2 = 1, b = 0$.

			(x_1, x_2)	Target y (Target)
			$(0, 0)$	0
			$(0, 1)$	0
(x_1, x_2)	y	$z = \omega_1 x_1 + \omega_2 x_2 + b$	$\hat{y}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$	$Error = y - \hat{y}$
$(0, 0)$	0	$z = 1 \cdot 0 + 1 \cdot 0 + 0 = 0$	1 (<i>wrong</i>)	$0 - 1 = -1$
Update: $w_1 := 1 + 1 \cdot (-1) \cdot 0 = 1$			$w_2 := 1 + 1 \cdot (-1) \cdot 0 = 1$	$b := 0 + 1 \cdot (-1) = -1$
$(0, 1)$	0	$z = 1 \cdot 0 + 1 \cdot 1 + 0 = 1$	$\hat{y} = 1$ (<i>wrong</i>)	$0 - 1 = -1$
Update: $w_1 := 1 + 1 \cdot (-1) \cdot 0 = 1$			$w_2 := 1 + 1 \cdot (-1) \cdot 1 = 0$	$b := -1 + 1 \cdot (-1) = -2$



Common Activation Functions

Epoch 2:

End Epoch 1: $w_1 = 1, w_2 = 1, b = 0$.

(x_1, x_2)	Target y (Target)
$(0, 0)$	0
$(0, 1)$	0
$(1, 0)$	0
$(1, 1)$	1

(x_1, x_2)	y	$z = \omega_1 x_1 + \omega_2 x_2 + b$	$\hat{y}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$	$Error = y - \hat{y}$
$(1, 0)$	0	$z = 1 \cdot 1 + 0 \cdot 0 + (-2) = -1$	$\hat{y} = 0$ (<i>correct</i>) (<i>target = 0</i>) $\rightarrow$ <i>ok</i> $\rightarrow$ No update	$0 - 0 = 0$
$(1, 1)$	1	$z = 1 \cdot 1 + 0 \cdot 1 + (-2) = -1$	0 (<i>Wrong</i>)	$1 - 0 = 1$
Update: $w_1 := 1 + 1 \cdot (1) \cdot 1 = 2$			$w_2 := 0 + 1 \cdot (1) \cdot 1 = 1$	$b := -2 + 1 \cdot (1) = -1$

End Epoch 2: $w_1 = 2, w_2 = 1, b = -1$



Common Activation Functions

Epoch 3

End Epoch 2: $w_1 = 2, w_2 = 1, b = -1$

- $(0,0)$: $z = 2.0 + 1.0 + (-1) = -1 \Rightarrow \hat{y} = 0. (target = 0) \rightarrow ok.$
- $(0,1)$: $z = 2.0 + 1.1 + (-1) = 0 \Rightarrow \hat{y} = 1. target = 0 \rightarrow error = -1.$

Update:

- $w_1 := 2 + 1 \cdot (-1) \cdot 0 = 2, w_2 := 1 + 1 \cdot (-1) \cdot 1 = 0, b := -1 + 1 \cdot (-1) = -2. \Rightarrow w = [2,0], b = -2.$
- $(1,0)$: $z = 2.1 + 1.0 + (-2) = 0 \Rightarrow \hat{y} = 1. target = 0 \rightarrow error = -1.$

Update:

- $w_1 := 2 + 1 \cdot (-2) \cdot 1 = 0, w_2 := 0 + 1 \cdot (-2) \cdot 0 = 0, b := -2 + 1 \cdot (-1) = -3 \Rightarrow w = [0,0], b = -3.$
- $(1,1)$: $z = 0.1 + 0.1 + (-3) = -2.8 \Rightarrow \hat{y} = 0. target = 1 \rightarrow error = +1.$

Update:

- $w_1 := 0 + 1 \cdot (+1) \cdot 1 = 1, w_2 := 0 + 1 \cdot (1) \cdot 1 = 1, b := -3 + 1 \cdot (1) = -2.$

End Epoch 3: $w = [2, 1], b = -2.$



Common Activation Functions

Epoch 4

End Epoch 3: $w_1 = 2, w_2 = 1, b = -2$

- $(0,0): z = 2.0 + 1.0 + (-2) = -2 \Rightarrow \hat{y} = 0. (target = 0) \rightarrow ok.$
- $(0,1): z = 2.0 + 1.1 + (-2) = -1 \Rightarrow \hat{y} = 0. (target = 0) \rightarrow ok.$
- $(1,0): z = 2.1 + 1.0 + (-2) = 0 \Rightarrow \hat{y} = 1. target = 0 \rightarrow error = -1.$

Update:

- $w_1 := 2 + 1 \cdot (-1) \cdot 1 = 1, w_2 := 1 + 1 \cdot (-1) \cdot 0 = 1, b := -2 + 1 \cdot (-1) = -3. \Rightarrow w = [1,1], b = -3.$
- $(1,1): z = 1.1 + 1.1 + (-3) = -1 \Rightarrow \hat{y} = 0. target = 1 \rightarrow error = +1.$

Update:

- $w_1 := 1 + 1 \cdot (1) \cdot 1 = 2, w_2 := 1 + 1 \cdot (1) \cdot 1 = 2, b := -3 + 1 \cdot (1) = -2$

End Epoch 4: $w = [2, 2], b = -2.$



Common Activation Functions

End Epoch 4: $w_1 = 2, w_2 = 2, b = -2$

- **Epoch 5**
- $(0,0): z = 2.0 + 2.0 + (-2) = -2 \Rightarrow 0. (target = 0) \rightarrow ok.$
- $(0,1): z = 2.0 + 2.1 + (-2) = 0 \Rightarrow \hat{y} = 1. target = 0 \rightarrow error = -1.$

Update:

- $w_1 := 2 + 1 \cdot (-1) \cdot 0 = 2, w_2 := 2 + 1 \cdot (-1) \cdot 1 = 1, b := -2 + 1 \cdot (-1) = -3 \Rightarrow w = [2,1], b = -3.$
- $(1,0): z = 2.1 + 1.0 + (-3) = -1 \Rightarrow 0. (target = 0) \rightarrow ok.$
- $(1,1): z = 2.1 + 1.1 + (-3) = 0 \Rightarrow \hat{y} = 1. target = 1 \rightarrow ok.$

→ End Epoch 5: $w = [2,1], b = -3.$



Common Activation Functions

End Epoch 5: $w_1 = 2, w_2 = 1, b = -3$

Final check (converged): With $w = [2, 1]$, $b = -3$,

- $(0,0): z = 2.0 + 1.0 + (-3) = -3 \Rightarrow \hat{y} = 0$ (*target* = 0) $\rightarrow ok$.
- $(0,1): z = 2.0 + 1.1 + (-3) = -2 \Rightarrow \hat{y} = 0$ (*target* = 0) $\rightarrow ok$.
- $(1,0): z = 2.1 + 1.0 + (-3) = -1 \Rightarrow \hat{y} = 0$ (*target* = 0) $\rightarrow ok$.
- $(1,1): z = 2.1 + 1.1 + (-3) = 0 \Rightarrow \hat{y} = 1$ (*target* = 1) $\rightarrow ok$.

All four are correct $\rightarrow$ perceptron has converged.



Common Activation Functions

Result & notes

End Epoch 5: $w_1 = 2, w_2 = 1, b = -3$

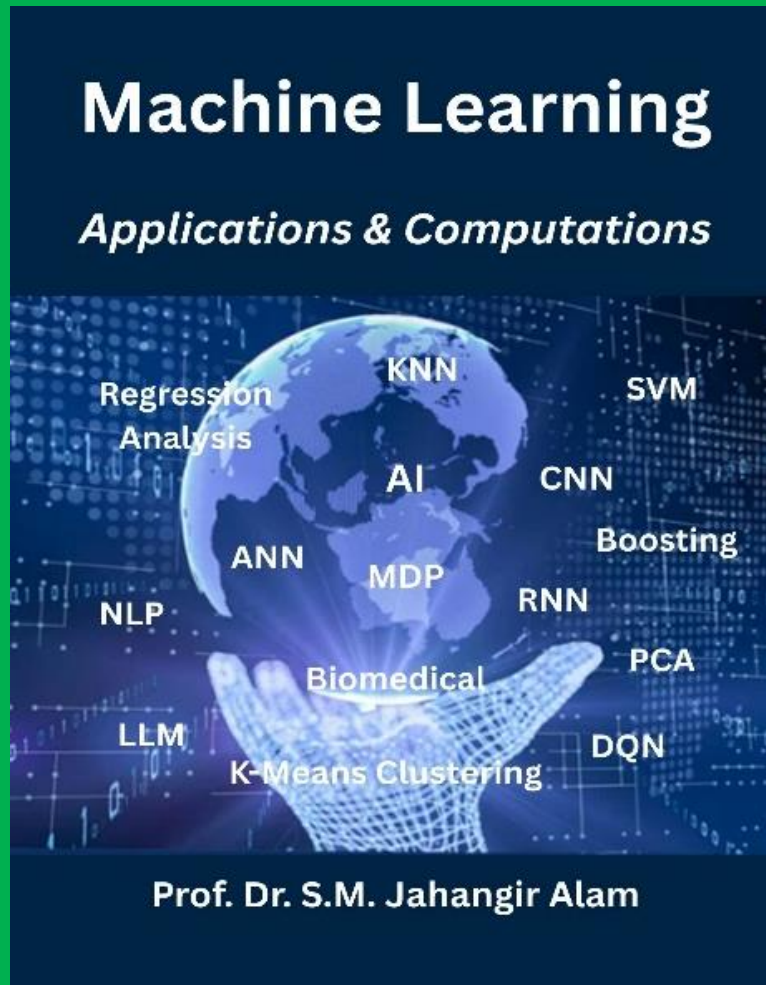
- Final weights: $w_1 = 2, w_2 = 1, b = -3$.
- Decision condition:

$$\hat{y}(z) = \begin{cases} 1 & z = \omega \cdot x + b \geq 0 \\ 0 & z < 0 \end{cases} \Rightarrow \hat{y}(z)$$
$$= \begin{cases} 1 & z = 2x_1 + 1x_2 - 3 \geq 0 \\ 0 & z < 0 \end{cases}$$

- The perceptron learning rule converged because the AND function is linearly separable.
- Important detail: ***step*(0) = 1** in your definition — that affected initial updates and boundary points.
- If the step at zero were defined as 0, intermediate updates would differ but convergence still occurs with appropriate ordering/learning rate.

Lecture 3.2

Neural Networks (NN)



Instructor Name:

Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST



Backpropagation in Machine Learning

Backpropagation is an algorithm used to train neural networks by updating weights to minimize a loss function. It is the core of supervised learning for deep networks.

Key idea:

- Compute error at the output
- Propagate this error backwards through the network
- Adjust weights and biases using gradient descent

Applications

- Deep Learning: CNN (images), RNN/LSTM (sequences), Transformers (NLP)
- Regression and classification tasks
- Reinforcement learning (backprop through policy/value networks)

Backpropagation in Neural Network

- *Backpropagation* is also known as "*Backward Propagation of Errors*" and it is a method used to train neural network.
- Its goal is to reduce the difference between the model's predicted output and the actual output by adjusting the weights and biases in the network.
- backpropagation is, why it is crucial in NN/ML and how it works.

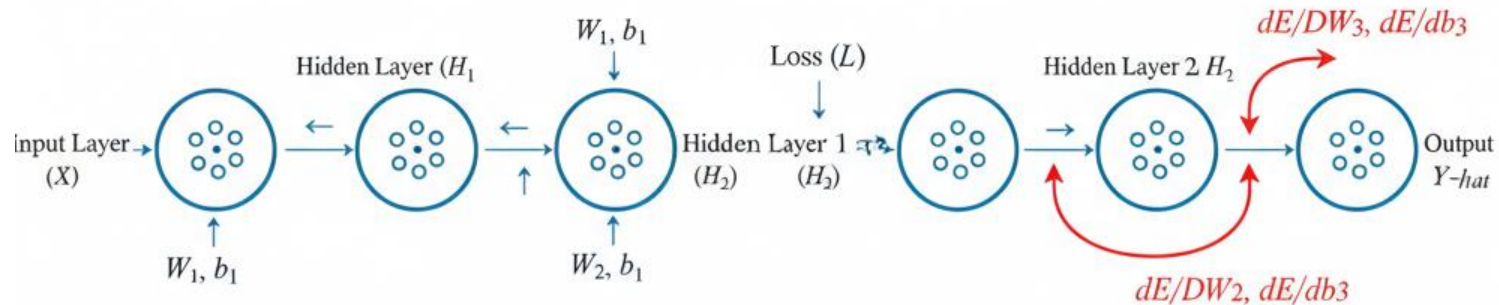
Backpropagation in Neural Network

- **Backpropagation** is a technique used in deep learning to train artificial neural networks particularly feed-forward networks. It works iteratively to adjust weights and bias to minimize the cost function.
- In each epoch the model adapts these parameters reducing loss by following the error gradient.
- Backpropagation often uses optimization algorithms like gradient descent or stochastic gradient descent.
- The algorithm computes the gradient using the chain rule from calculus allowing it to effectively navigate complex layers in the neural network to minimize the cost function.

Backpropagation in Neural Network

FORWARD PROPAGATION

BACKWARD PROPAGATION



$$Z1 = X * W1 + b1$$

$$Z1 = f(Z1)$$

$$A1 = A1 * W2 + b2$$

$$A2 = f(Z1)$$

$$Z3 = A2 * W3 + b3$$

$$Y-hat = g(Z3)$$

$$dE/dY-hat = Y-hat - Y \text{ (True)}$$

$$dE/dZ3 = A2^T * g' (Z3)$$

$$dE/dW3 = dE/dZ3 * dE/db3^T$$

$$sum(dZ3)$$

$$dE/dA2 = dE/dZ3 * f' A^T$$

$$dE/AZ_2 = sum(f23)$$

UPDATE WEIGHTS: $W = W - \text{learning_rate} * dE/dW$

Backpropagation in Neural Network

Initial Calculation: The weighted sum at each node is calculated using,

$$z_j = \sum(w_{i,j} * x_i)$$

Where, a_j is the weighted sum of all the inputs and weights at each node, $w_{i,j}$ represents the weights between the i^{th} input and the j^{th} neuron, x_i represents the value of the i^{th} input.

O (output): After applying the activation function to a , we get the output of the neuron:

$$O_j = \text{activation function}(z_j)$$

Sigmoid Function: The sigmoid function returns a value between 0 and 1, introducing non-linearity into the model.

$$y_j = \frac{1}{1 + e^{-z_j}}$$

Computing outputs: At h_1 and h_2 node,

Backpropagation in Neural Network

Error calculation: The actual output is y_{target} but the calculated output is obtained y_j . Thus, the error e_j can be calculated as,

$$e_j = y_{target} - y_j$$

This error can be used for backpropagating.

Step 1: Forward Pass: The network has no hidden layer, 2 inputs, 1 output neuron with sigmoid activation:

$$z = \omega_1 x_1 + \omega_2 x_2 + b$$

$$\therefore y_{pred} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Backpropagation in Neural Network

Step 2: Compute Error: Using mean squared error (MSE),

$$Error, E = \frac{1}{2} (y - \hat{y})^2$$

Step 3: Backward Pass (Backpropagation)

We'll compute the gradient of the error with respect to each weight.

Derivative of sigmoid:

$$\sigma'(z) = \hat{y}(1 - \hat{y})$$

Gradient for each weight using chain rule,

$$\frac{\partial E}{\partial \omega_i} = (y - \hat{y})(-1) \cdot \sigma'(z) \cdot x_i$$

Backpropagation in Neural Network

Step 4: Update Weights: Using gradient descent,

$$\omega_i = \omega_i - \eta \cdot \frac{\partial E}{\partial b}$$

Calculating Gradients: The change in each weight is calculated as:

$$\Delta\omega_{ij} = \eta \times \delta_j \times O_j$$

Where, δ_j is the error term for each unit, η is the learning rate.

The output unit error: For O_j

$$\delta_j = y_j(1 - y_j)(y_{target} - y_j)$$

The hidden unit error: For h_i :

$$\delta_i = y_i(1 - y_i) \times w_{i,j} \times \delta_j$$

Weight updates

$$\Delta\omega_{ij} = \eta \times \delta_j \times O_j$$



Backpropagation in Machine Learning

Goal: Train a neural network to learn the AND logic gate.

Problem:

Input x_1, x_2	Output y
(1, 1)	1
(1, 0)	0
(0, 1)	0
(0, 0)	0

We'll train using just one data point:

Input: (1, 1), Target Output: 1

Network Architecture

- 2 inputs $\rightarrow$ 1 output neuron (no hidden layer)
- Activation function: Sigmoid
- Initial weights: $w_1=0.5$, $w_2=0.5$, Bias $b = -1$
- Learning rate $\eta = 0.1$



Backpropagation in Machine Learning

Input x_1, x_2	Output y
(1, 1)	1
(1, 0)	0
(0, 1)	0
(0, 0)	0

Step 1: Forward Pass

Inputs: $x_1 = 1, x_2 = 1$

Weighted sum:

$$z = \omega_1 x_1 + \omega_2 x_2 + b = (0.5)(1) + (0.5)(1) + (-1) = 1.0 - 1 = 0$$

$$\text{Apply sigmoid: } \hat{y} = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^0} = 0.5$$



Backpropagation in Machine Learning

Input x_1, x_2	Output y
(1, 1)	1
(1, 0)	0
(0, 1)	0
(0, 0)	0

Step 1: Forward Pass

Inputs: $x_1 = 1, x_2 = 1$

Weighted sum:

$$\begin{aligned} z &= \omega_1 x_1 + \omega_2 x_2 + b \\ &= (0.5)(1) + (0.5)(1) + (-1) = 1.0 - 1 = 0 \end{aligned}$$

$$\text{Apply sigmoid: } \hat{y} = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^0} = 0.5$$

Step 2: Compute Error

Using squared error:

$$\begin{aligned} \text{Error, } E &= \frac{1}{2} (y - \hat{y})^2 \\ &= \frac{1}{2} (1 - 0.5)^2 = 0.125 \end{aligned}$$



Backpropagation in Machine Learning

	$z = \omega_1 x_1 + \omega_2 x_2 + b$	$y_{pred} = \hat{y} = \sigma(z)$ $= \frac{1}{1 + e^{-z}}$	$MSE, E = \frac{1}{2} (y - \hat{y})^2$
(1,1)	z $= (0.5)(1) + (0.5)(1)$ $+ (-1) = 0$	$\hat{y} = \frac{1}{1 + e^0} = 0.5$	$\frac{1}{2} (1 - 0.5)^2 = 0.125$
(1,0)	z $= (0.5)(1) + (0.5)(0)$ $+ (-1) = -0.5$	$\hat{y} = \frac{1}{1 + e^{0.5}}$ $= 0.3775$	$\frac{1}{2} (1 - 0.3775)^2$ $= 0.1937$
(0,1)	z $= (0.5)(0) + (0.5)(1)$ $+ (-1) = -0.5$	$\hat{y} = \frac{1}{1 + e^{0.5}}$ $= 0.3775$	$\frac{1}{2} (1 - 0.3775)^2$ $= 0.1937$
(0,0)	z $= (0.5)(0) + (0.5)(0)$ $+ (-1) = -1$	$\hat{y} = \frac{1}{1 + e^1} = 0.2689$	$\frac{1}{2} (1 - 0.2689)^2$ $= 0.2673$



Backpropagation in Machine Learning

Input x_1, x_2	Output y
(1, 1)	1
(1, 0)	0
(0, 1)	0
(0, 0)	0

Step 3: Backward Pass (Backpropagation)

Compute the gradient of the error with respect to each weight.

Derivative of sigmoid:

$$\sigma'(z) = \hat{y}(1 - \hat{y}) = 0.5(1 - 0.5) = 0.25$$

Gradient for each weight:

Using chain rule: $\frac{\partial E}{\partial \omega_i} = (y - \hat{y})(-1) \cdot \sigma'(z) \cdot x_i$



Backpropagation in Machine Learning

Input x_1, x_2	Output y
(1, 1)	1
(1, 0)	0
(0, 1)	0
(0, 0)	0

For ω_1 : $\frac{\partial E}{\partial \omega_1} = (1 - 0.5)(-1) \times 0.25 \times 1 = -0.125$

For ω_2 (same input): $\frac{\partial E}{\partial \omega_2} = -0.125$

For bias b : $\frac{\partial E}{\partial b} = -0.125 \times 1 = -0.125$



Backpropagation in Machine Learning

Input x_1, x_2	Output y
(1, 1)	1
(1, 0)	0
(0, 1)	0
(0, 0)	0

Step 4: Update Weights

Using gradient descent:

$$\omega_i = \omega_i - \eta \cdot \frac{\partial E}{\partial b}$$

Update weights:

- $\omega_1 = 0.5 - 0.1(-0.125) = 0.5125$
- $\omega_2 = 0.5 + 0.0125 = 0.5125$
- $b = -1 + 0.0125 = -0.9875$



Backpropagation in Machine Learning

	(1,1)
$\sigma'(z) = \hat{y}(1 - \hat{y})$	$\hat{y}(1 - \hat{y}) = 0.5(1 - 0.5) = 0.25$
$\frac{\partial E}{\partial \omega_i}$ $= (y - \hat{y})(-1)$ $\cdot \sigma'(z) \cdot x_i$	For ω_1: $\frac{\partial E}{\partial \omega_1} = (1 - 0.5)(-1) \times 0.25 \times 1 = -0.125$ For ω_2: $\frac{\partial E}{\partial \omega_1} = (1 - 0.5)(-1) \times 0.25 \times 1 = -0.125$ For bias b: $\frac{\partial E}{\partial b} = -0.125 \times 1 = -0.125$
Update Weights $\omega_i = \omega_i - \eta \cdot \frac{\partial E}{\partial \omega_i}$	$\omega_1 = 0.5 - 0.1(-0.125) = 0.5125$ $\omega_2 = 0.5 + 0.0125 = 0.5125$ $b = -1 + 0.0125 = -0.9875$



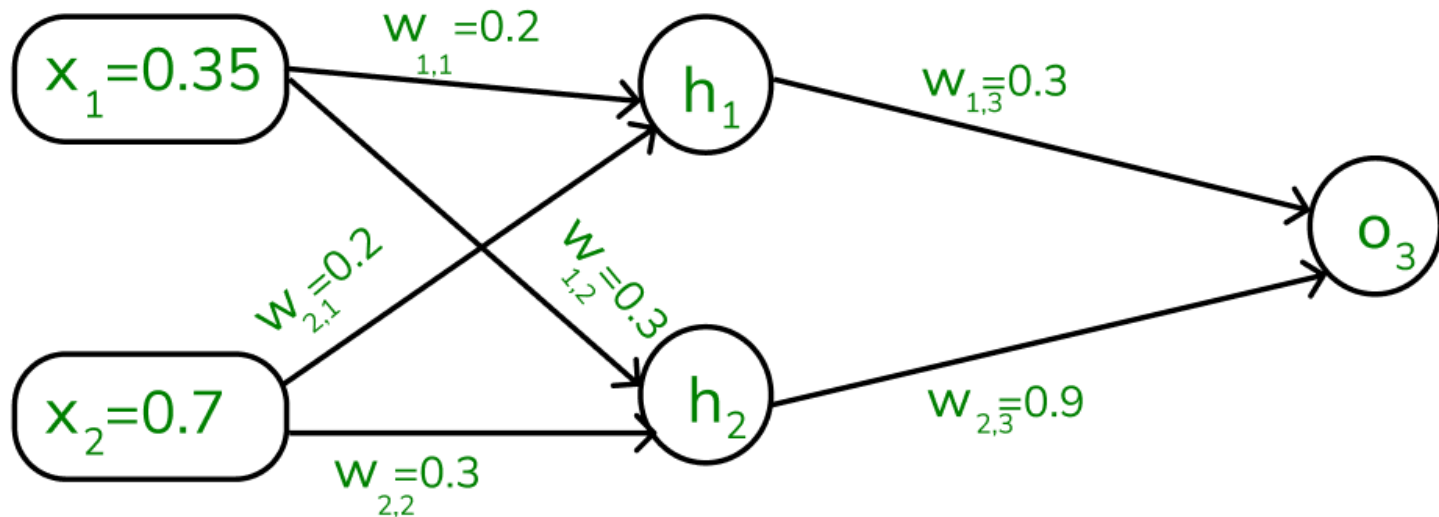
Backpropagation in Machine Learning

Summary Training Step:

Parameter	Old Value	Gradient	New Value
ω_1	0.5	-0.125	0.5125
ω_2	0.5	-0.125	0.5125
b	-1	-0.125	-0.9875

Example of Backpropagation in Machine Learning

Let's walk through an example of backpropagation in machine learning. Assume the neurons use the sigmoid activation function for the forward and backward pass. The target output is 0.5, and the learning rate is 1.



Forward Propagation Calculation

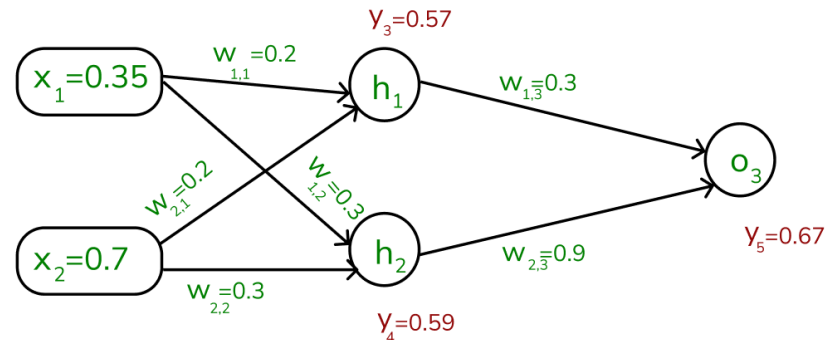
$$z_j = \sum(w_{i,j} * x_i)$$

O_j = activation function (z_j)

- **Sigmoid Function:** The sigmoid function returns a value between 0 and 1, introducing non-linearity into the model.

$$y_j = \frac{1}{1 + e^{-z_j}}$$

Forward Propagation Calculation



Computing outputs

$$z_j = \sum(w_{i,j} * x_i)$$

$$y_j = f(z_j) = \frac{1}{1 + e^{-z_j}}$$

$$\begin{aligned} z_1 &= w_{1,1}x_1 + w_{2,1}x_2 \\ &= 0.2 \times 0.35 + 0.2 \times 0.7 = 0.21 \end{aligned}$$

$$y_3 = \frac{1}{1 + e^{-0.21}} = 0.56$$

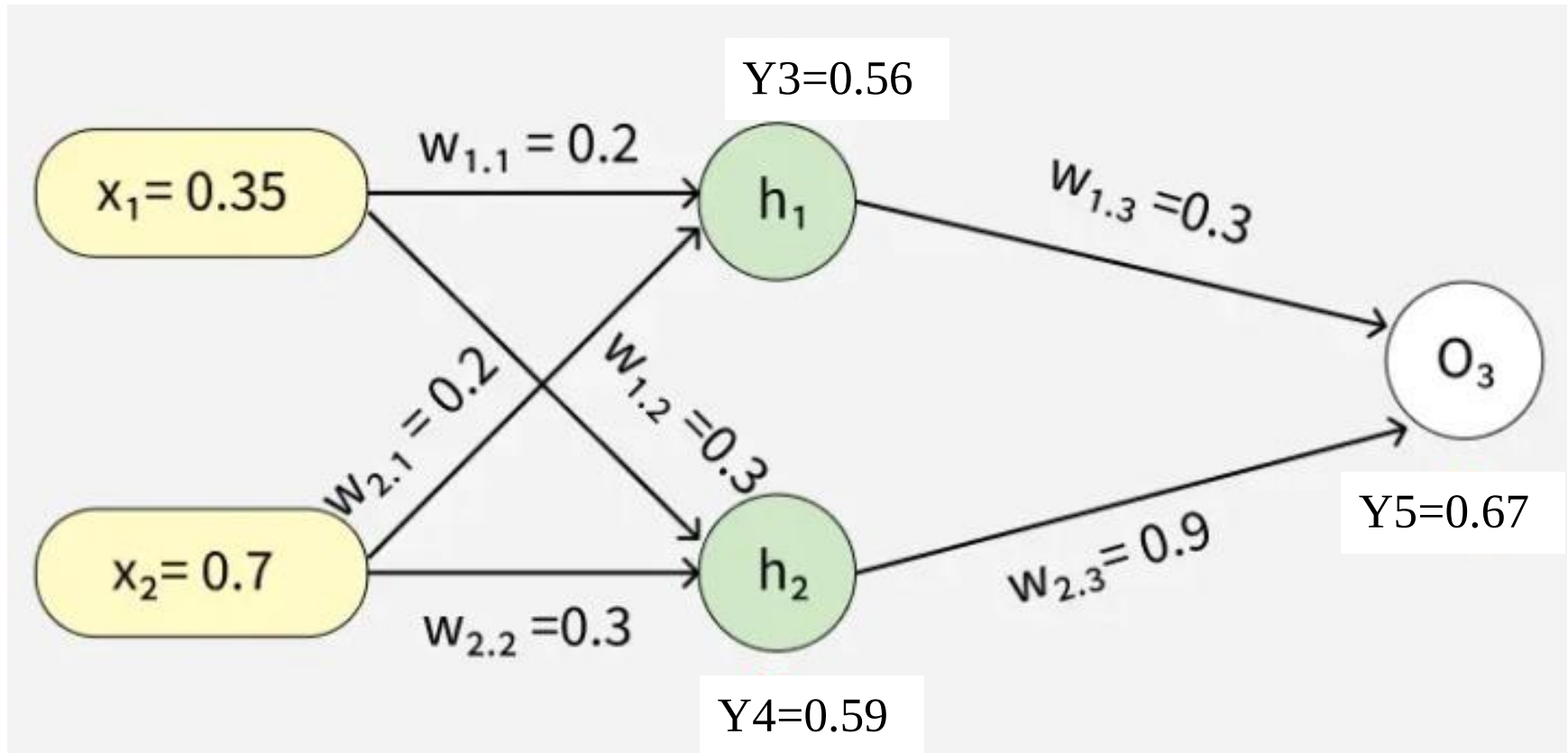
$$\begin{aligned} z_2 &= w_{1,2}x_1 + w_{2,2}x_2 \\ &= 0.3 \times 0.35 + 0.3 \times 0.7 = 0.315 \end{aligned}$$

$$y_4 = \frac{1}{1 + e^{-0.315}} = 0.59$$

$$\begin{aligned} z_3 &= w_{1,3}y_1 + w_{2,3}y_2 \\ &= 0.3 \times 0.56 + 0.3 \times 0.59 = 0.702 \end{aligned}$$

$$\begin{aligned} y_5 &= o_3 = \frac{1}{1 + e^{-0.702}} \\ &= 0.67 \end{aligned}$$

Forward Propagation Calculation

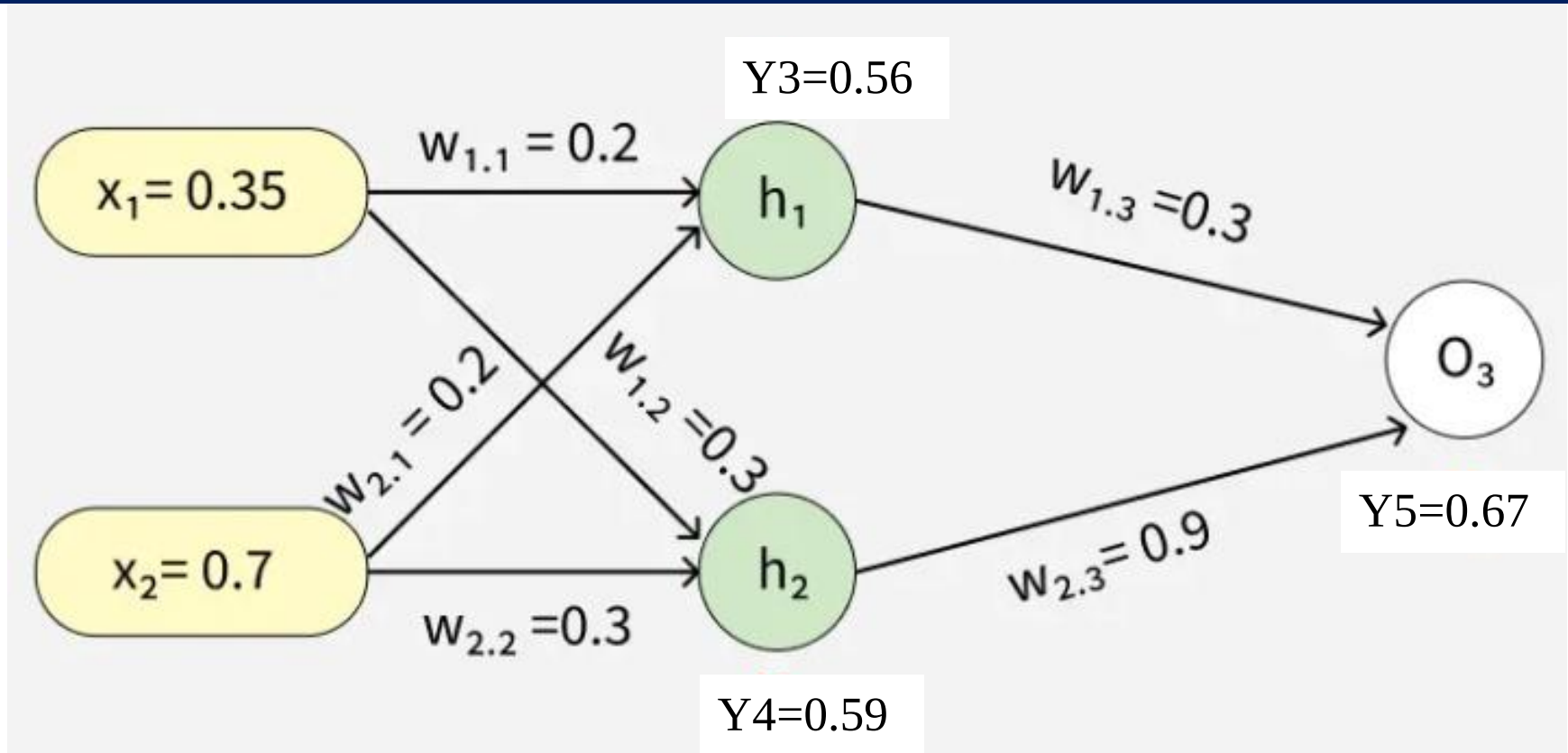


Error calculation: The actual output is $y_{target} = 0.5$ but the calculated output is obtained $y_3 = 0.67$. Thus, the error e_j can be calculated as,

$$e_j = y_{target} - y_3 = 0.5 - 0.67 = -0.17$$

This error can be used for backpropagating.

Forward Propagation Calculation

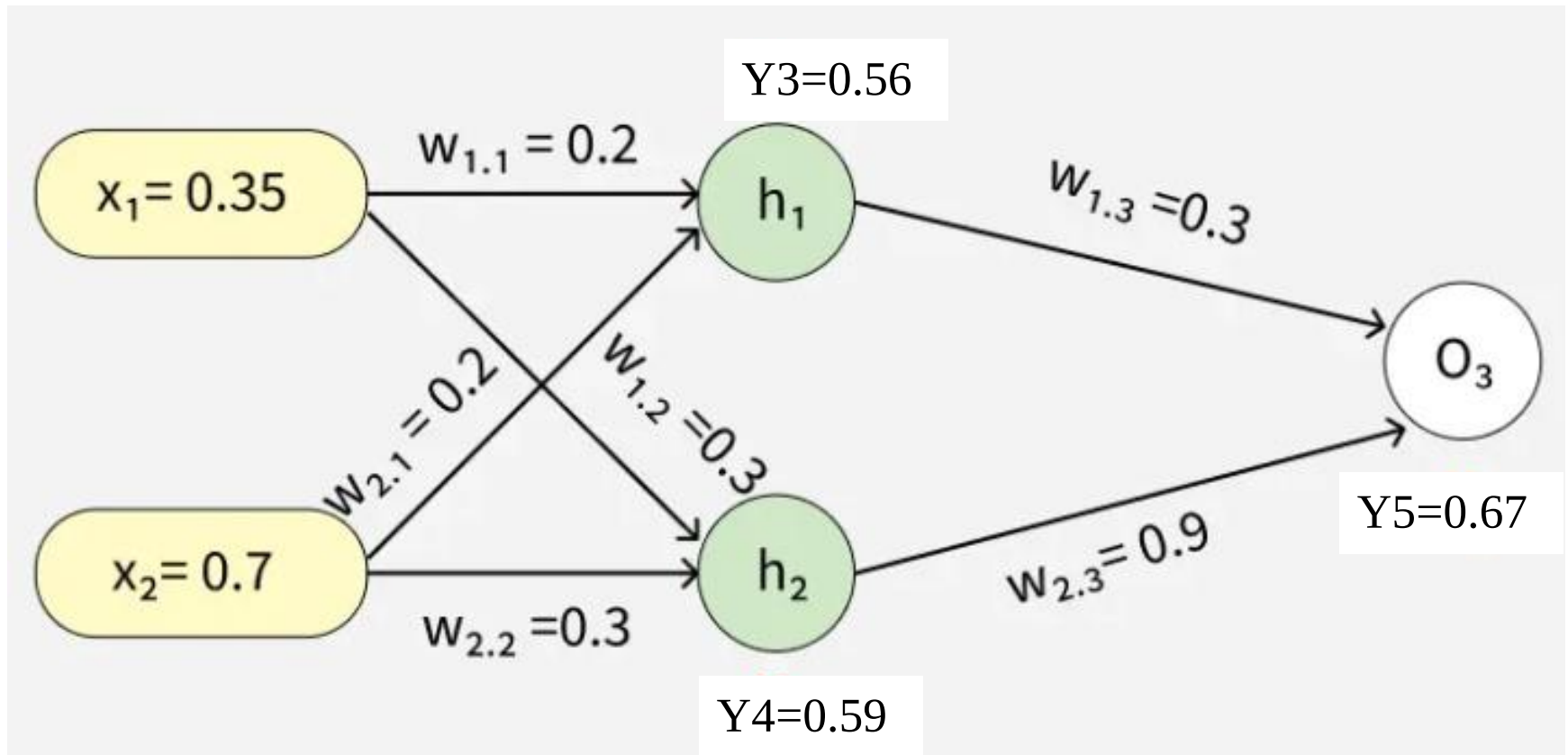


1. Calculating Gradients: The change in each weight is calculated as:

$$\Delta\omega_{ij} = \eta \times \delta_j \times O_j$$

Where, δ_j is the error term for each unit, η is the learning rate.

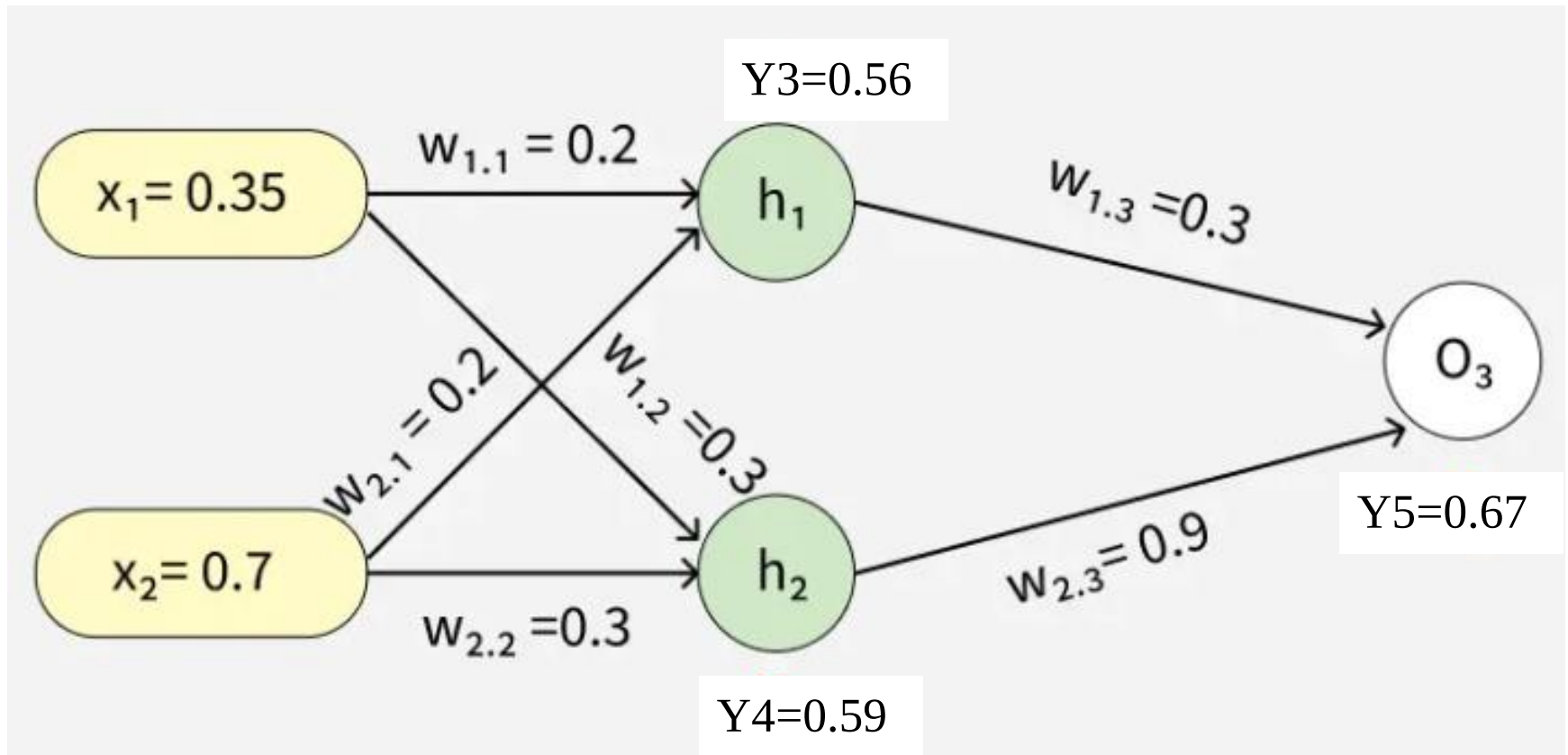
Backpropagation Calculation



2. The output unit error: For O_j

$$\delta_5 = y_5(1 - y_5)(y_{target} - y_5) = 0.67(1 - 0.67)(0.5 - 0.67) \\ = -0.0376$$

Backpropagation Calculation

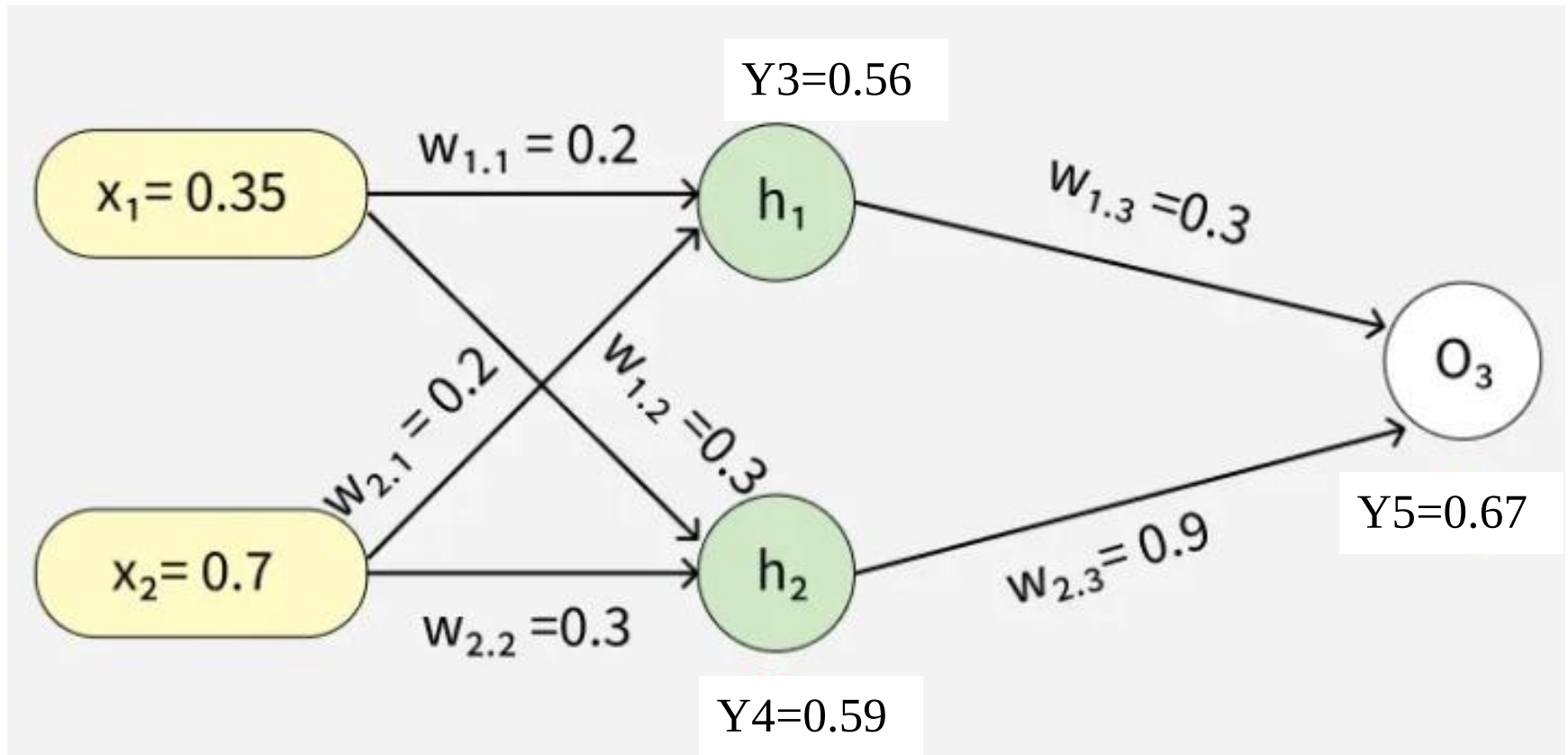


3. The hidden unit error:

For h_1 :

$$\begin{aligned}\delta_3 &= y_3(1 - y_3) \times w_{1,3} \times \delta_5 \\ &= 0.56(1 - 0.56) \times 0.3 \times (-0.0376) = -0.0027\end{aligned}$$

Backpropagation Calculation



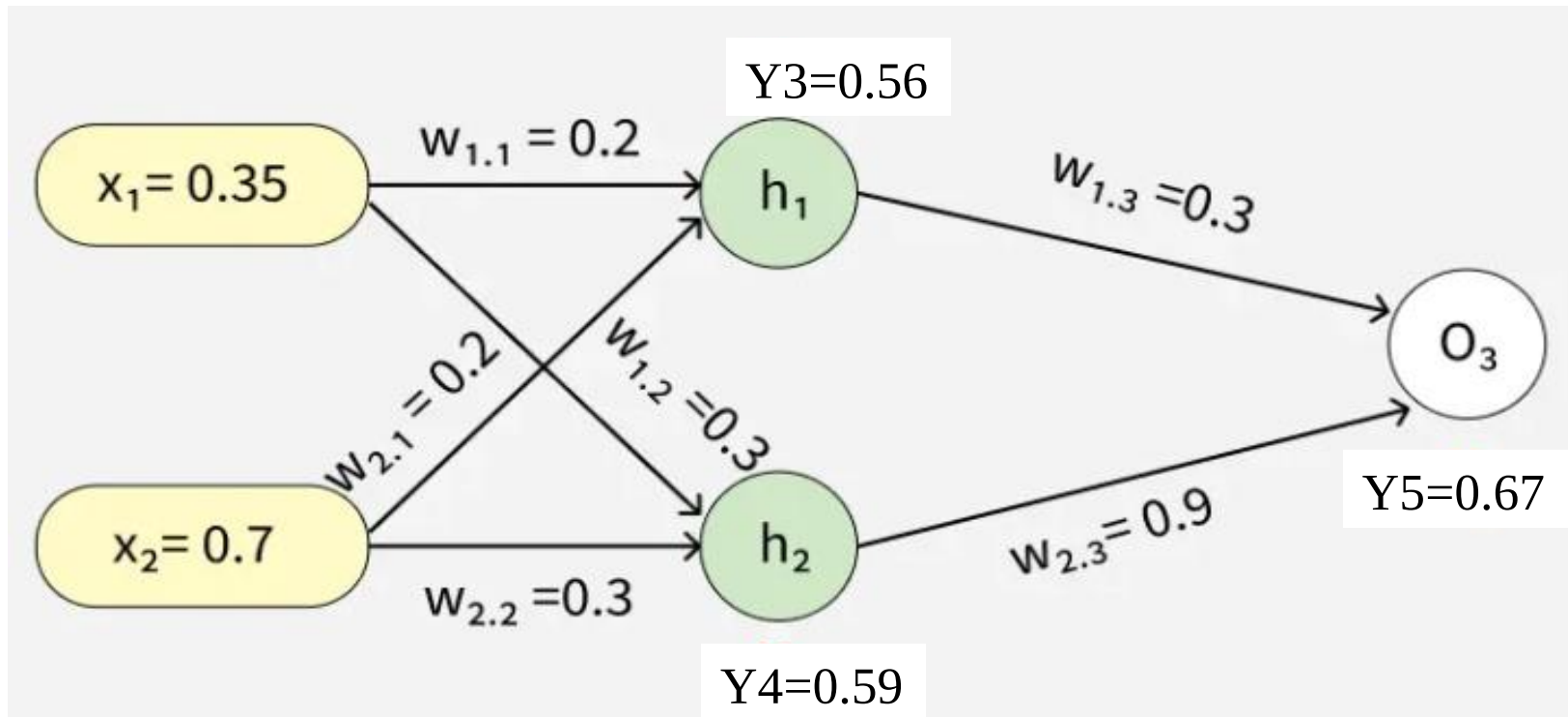
For h_2 :

$$\begin{aligned}\delta_4 &= y_4(1 - y_4) \times w_{2,3} \times \delta_5 \\ &= 0.59(1 - 0.59) \times 0.9 \times (-0.0376) = -0.0819\end{aligned}$$

Backpropagation Calculation

4. Weight updates

For the weights from hidden to output layer	New weight
$\Delta\omega_{2,3} = \eta \times \delta_5 \times Y_4$ $= 1 \times (-0.0376) \times 0.59 = -0.02218$	$\omega_{2,3} (new) = -0.02218 + 0.9$ $= 0.8778$
$\Delta\omega_{1,3} = \eta \times \delta_5 \times Y_3$ $= 1 \times (-0.0376) \times 0.56 = -0.01056$	$\omega_{1,3} (new) = -0.02519 + 0.3$ $= 0.278944$



Backpropagation Calculation

4. Weight updates

For the weights from input to hidden layer

$$\begin{aligned}\Delta\omega_{1,1} &= \eta \times \delta_3 \times \mathbf{x}_1 \\ &= 1 \times (-0.0027) \times 0.35 = -0.000945\end{aligned}$$

$$\begin{aligned}\omega_{1,1} (new) &= -0.00945 + 0.2 \\ &= 0.199055\end{aligned}$$

$$\begin{aligned}\Delta\omega_{1,2} &= \eta \times \delta_4 \times \mathbf{x}_1 \\ &= 1 \times (-0.0819) \times 0.35 = -0.028665\end{aligned}$$

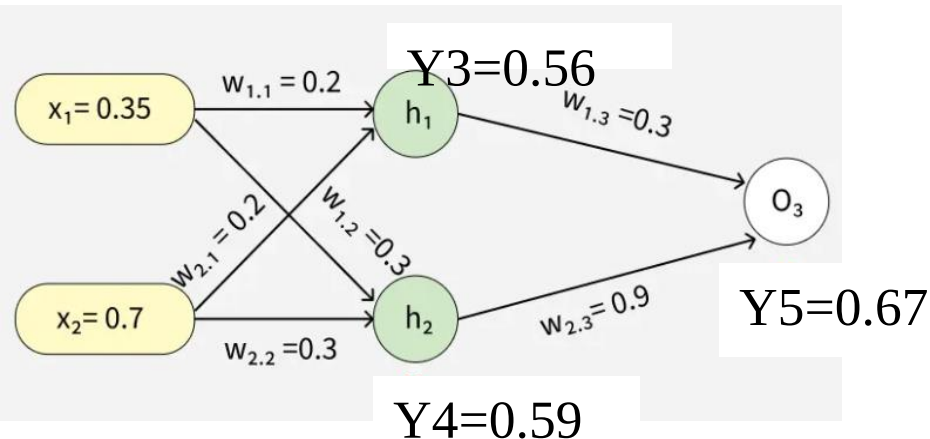
$$\begin{aligned}\omega_{1,2} (new) &= -0.028665 + 0.3 \\ &= 0.271335\end{aligned}$$

$$\begin{aligned}\Delta\omega_{2,1} &= \eta \times \delta_3 \times \mathbf{x}_2 \\ &= 1 \times (-0.0027) \times 0.7 = -0.00189\end{aligned}$$

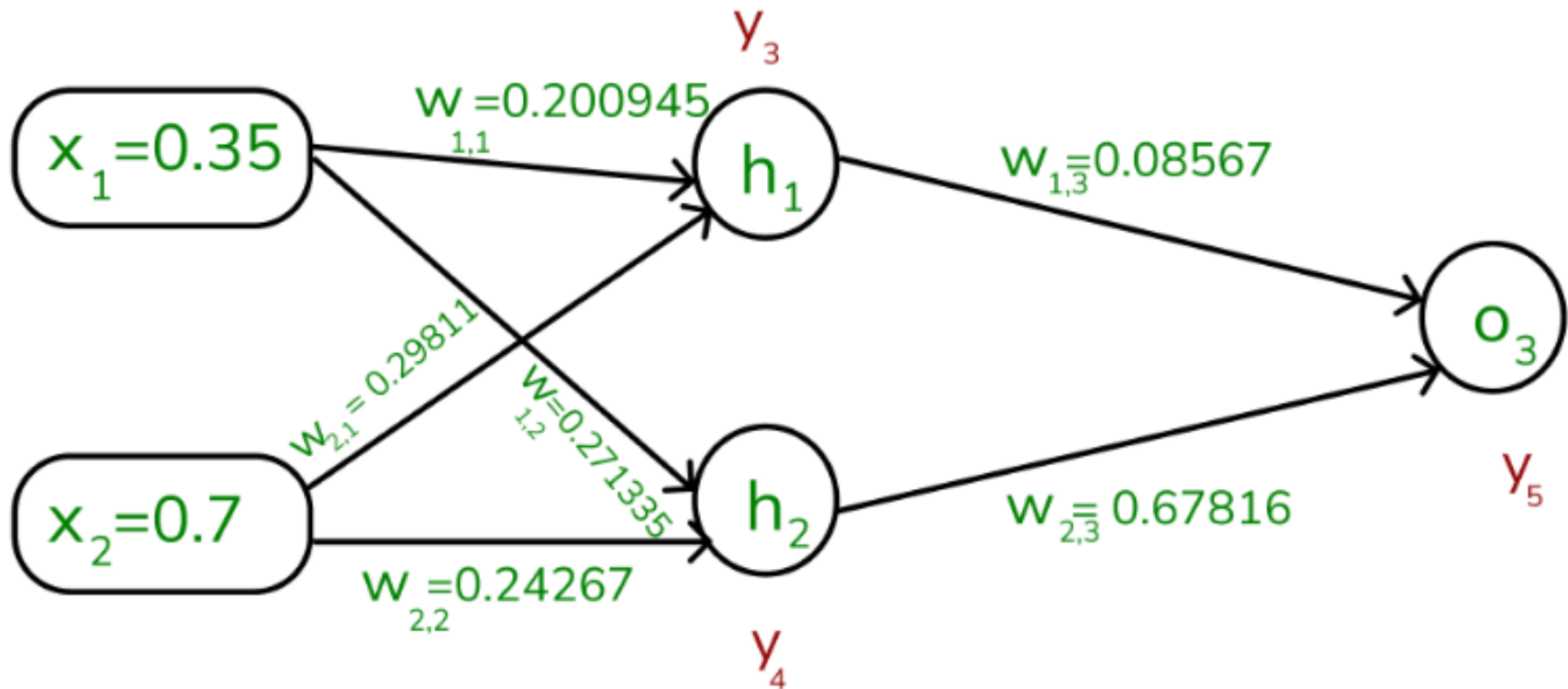
$$\begin{aligned}\omega_{2,1} (new) &= -0.00189 + 0.2 \\ &= 0.19811\end{aligned}$$

$$\begin{aligned}\Delta\omega_{2,2} &= \eta \times \delta_4 \times \mathbf{x}_2 \\ &= 1 \times (-0.0819) \times 0.7 = -0.05733\end{aligned}$$

$$\begin{aligned}\omega_{2,2} (new) &= -0.05733 + 0.3 \\ &= 0.24267\end{aligned}$$

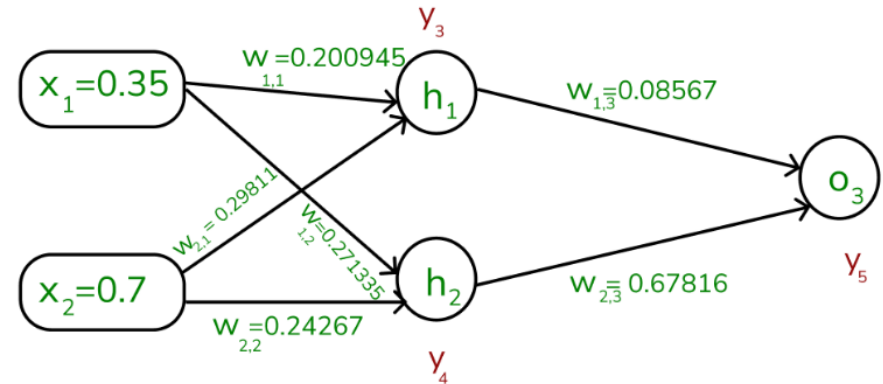


Backpropagation Calculation



After updating the weights, the forward pass is repeated yielding:

Backpropagation Calculation



$$z_j = \sum(w_{i,j} * x_i)$$

$$y_j = f(z_j) = \frac{1}{1 + e^{-z_j}}$$

$$\begin{aligned} z_3 &= w_{1,1}x_1 + w_{2,1}x_2 \\ &= 0.199055 \times 0.35 + 0.19811 \times 0.7 \\ &= 0.2083 \end{aligned}$$

$$\begin{aligned} y_3 &= \frac{1}{1 + e^{-0.2083}} \\ &= 0.55 \end{aligned}$$

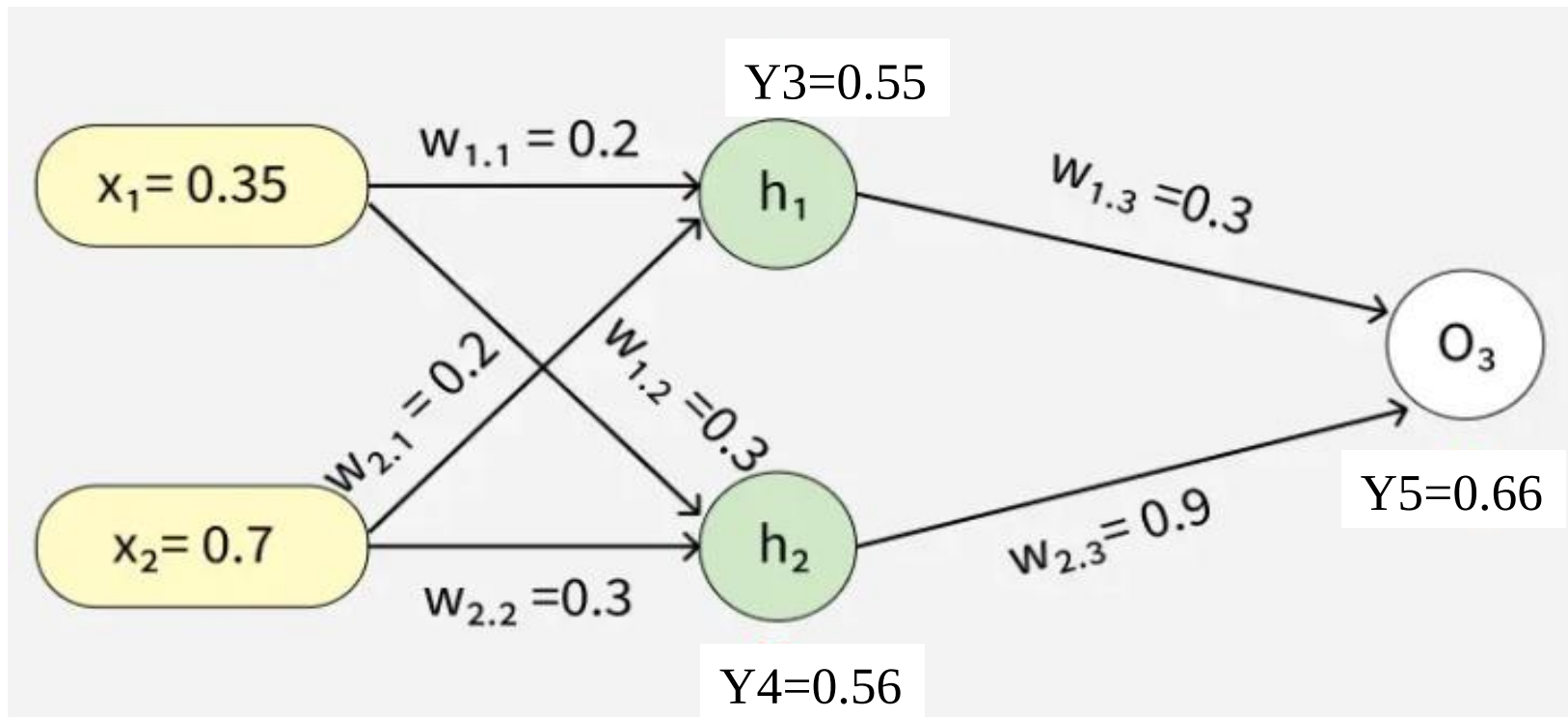
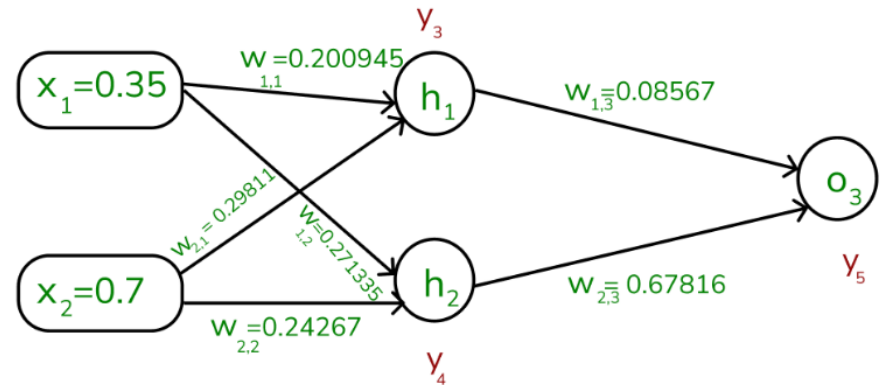
$$\begin{aligned} z_4 &= w_{1,2}x_1 + w_{2,2}x_2 \\ &= 0.271335 \times 0.35 + 0.24267 \times 0.7 \\ &= 0.2648 \end{aligned}$$

$$\begin{aligned} y_4 &= \frac{1}{1 + e^{-0.2648}} \\ &= 0.57 \end{aligned}$$

$$\begin{aligned} z_5 &= w_{1,3}y_1 + w_{2,3}y_2 \\ &= 0.277816 \times 0.56 + 0.877816 \times 0.59 \\ &= 0.6734 \end{aligned}$$

$$\begin{aligned} y_5 &= \frac{1}{1 + e^{-0.6734}} \\ &= 0.66 \end{aligned}$$

Backpropagation Calculation



Backpropagation Calculation

- Since $y_5 = 0.66$ is still not the target output the process of calculating the error and backpropagating continues until the desired output is reached.
- This process demonstrates how backpropagation iteratively updates weights by minimizing errors until the network accurately predicts the output.

$$e_j = y_{target} - y_5 = 0.5 - 0.66 = -0.16$$

- This process is said to be continued until the actual output is gained by the neural network.

Thank You